Synthos Complete API Architecture & Frontend Design

Microservices Communication Blueprint

Last Updated: October 22, 2025

Version: 1.0

Classification: Technical Architecture Document

I. MICROSERVICES ARCHITECTURE OVERVIEW

1.1 Service Communication Map

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                    API GATEWAY (Go)                                                          │
│                  - REST API (Customer-facing)                                                │
│                  - gRPC Gateway (Internal services)                                          │
│                  - Authentication/Authorization                                              │
│                  - Rate limiting                                                             │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
                    │
                    │ gRPC
        ┌───────────┼──────────────────────────────────────────────────────────────────────┐
        │           │                                                                      │
        ▼           ▼                                                                      ▼
┌──────────────────────┐   ┌──────────────────────────────────────────────────────────┐   ┌──────────────────────┐
│ Job                  │   │ Validation                                               │   │ Collapse             │
│ Orchestrator         │◄──►│ Engine Service                                           │◄──►│ Engine               │
│ (Go)                 │   │ (Python)                                                 │   │ Service              │
│                      │   │                                                          │   │ (Python)             │
└──────────────────────┘   └──────────────────────────────────────────────────────────┘   └──────────────────────┘
        │                  │                                                                │
        │                  │                                                                │
        ▼                  ▼                                                                ▼
┌──────────────────────┐   ┌──────────────────────────────────────────────────────────┐   ┌──────────────────────┐
│ Data Service         │   │ Report Generator                                         │   │ Signature            │
│ (Go)                 │   │ (Python)                                                 │   │ Library              │
│                      │   │                                                          │   │ (Python)             │
└──────────────────────┘   └──────────────────────────────────────────────────────────┘   └──────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                    Data Layer (PostgreSQL + Redis + S3)                                      │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

1.2 Communication Protocols

External (Customer → API Gateway):

- Protocol: REST over HTTPS
- Format: JSON
- Authentication: JWT tokens
- Rate Limiting: Per customer tier

Internal (Service ↔ Service):

- Protocol: gRPC (high performance, type-safe)
- Format: Protocol Buffers
- Authentication: mTLS (mutual TLS)
- Timeout: 60s for sync, infinite for async

Async Communication:

- Message Queue: RabbitMQ
- Pattern: Pub/Sub for events, Work Queue for jobs
- Retry Policy: Exponential backoff (3 retries)
- Dead Letter Queue: For failed messages

II. API GATEWAY ROUTES (Customer-Facing REST API)

2.1 Authentication & User Management

Base URL: `https://api.synthos.ai/v1`

`POST /auth/register`

Purpose: Register new user account

Request:

```
json
```

```
{
  "email": "user@company.com",
  "password": "SecurePass123!",
  "company_name": "Acme AI Labs",
  "full_name": "John Doe"
}
```

Response:

```json
{
  "user_id": "usr_abc123",
  "email": "user@company.com",
  "company_id": "cmp_xyz789",
  "created_at": "2025-10-22T14:30:00Z"
}
```

POST /auth/login

Purpose: Authenticate user and get JWT token

Request:

```json
{
  "email": "user@company.com",
  "password": "SecurePass123!"
}
```

Response:

```json
```

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "user_id": "usr_abc123",
    "email": "user@company.com",
    "company_name": "Acme AI Labs",
    "subscription_tier": "professional"
  }
}
```

POST /auth/refresh

Purpose: Refresh access token using refresh token

Request:

```json
{
  "refresh_token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Response:

```json
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "expires_in": 3600
}
```

2.2 Dataset Management

POST /datasets/upload

Purpose: Initiate dataset upload and get signed URL

Request:

```json
```

```
{
  "filename": "training_data.csv",
  "file_size": 52428800,
  "file_type": "text/csv",
  "description": "Synthetic customer data for LLM training"
}
```

Response:

```json
{
  "dataset_id": "ds_def456",
  "upload_url": "https://s3.amazonaws.com/synthos-uploads/...",
  "upload_method": "multipart",
  "chunk_size": 10485760,
  "expires_in": 3600
}
```

Frontend Action: Use upload_url to stream file chunks directly to S3

POST /datasets/{dataset_id}/complete

Purpose: Mark upload as complete and trigger processing

Request:

```json
{
  "dataset_id": "ds_def456",
  "etag": "e47f38ca7b9c1f1cf3c31d28f69ba55b"
}
```

Response:

```json
```

```
{
  "dataset_id": "ds_def456",
  "status": "processing",
  "estimated_completion": "2025-10-22T16:00:00Z",
  "processing_stages": [
    {
      "stage": "ingestion",
      "status": "in_progress",
      "progress": 0
    },
    {
      "stage": "profiling",
      "status": "pending",
      "progress": 0
    }
  ]
}
```

GET /datasets/{dataset_id}

Purpose: Get dataset details and processing status

Response:

```
json
```

```
{
  "dataset_id": "ds_def456",
  "filename": "training_data.csv",
  "file_size": 52428800,
  "row_count": 500000000,
  "column_count": 50,
  "status": "processed",
  "uploaded_at": "2025-10-22T14:30:00Z",
  "processed_at": "2025-10-22T15:00:00Z",
  "metadata": {
    "data_quality_score": 87,
    "has_pii": false,
    "schema": [
      {
        "column_name": "user_id",
        "data_type": "string",
        "null_rate": 0.001,
        "unique_rate": 0.95
      }
    ]
  }
}
```

`GET /datasets`

Purpose: List all datasets for authenticated user

Query Parameters:

- `page` (default: 1)
- `page_size` (default: 20)
- `status` (filter: uploaded, processing, processed, failed)
- `sort_by` (default: created_at)

Response:

```
json
```

```
{
  "datasets": [
    {
      "dataset_id": "ds_def456",
      "filename": "training_data.csv",
      "row_count": 500000000,
      "status": "processed",
      "uploaded_at": "2025-10-22T14:30:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "page_size": 20,
    "total_count": 45,
    "total_pages": 3
  }
}
```

DELETE /datasets/{dataset_id}

Purpose: Delete dataset and all associated data

Response:

```
{
  "dataset_id": "ds_def456",
  "status": "deleted",
  "deleted_at": "2025-10-22T16:00:00Z"
}
```

2.3 Validation Jobs

POST /validations/create

Purpose: Create validation job for a dataset

Request:

```
```

```
{
  "dataset_id": "ds_def456",
  "validation_type": "full",
  "options": {
    "target_model_size": "1B",
    "target_architecture": "transformer",
    "priority": "standard",
    "enable_warranty": true
  }
}
```

Response:

```json
{
  "validation_id": "val_ghi789",
  "dataset_id": "ds_def456",
  "status": "queued",
  "estimated_completion": "2025-10-23T14:30:00Z",
  "estimated_cost": 35000,
  "stages": [
    {
      "stage": "diversity_analysis",
      "status": "pending",
      "estimated_duration": 14400
    },
    {
      "stage": "cascade_training",
      "status": "pending",
      "estimated_duration": 108000
    },
    {
      "stage": "collapse_detection",
      "status": "pending",
      "estimated_duration": 21600
    },
    {
      "stage": "report_generation",
      "status": "pending",
      "estimated_duration": 7200
    }
  ]
}
```

GET /validations/{validation_id}

Purpose: Get validation job status and results

Response (In Progress):

```json
{
  "validation_id": "val_ghi789",
  "dataset_id": "ds_def456",
  "status": "running",
  "created_at": "2025-10-22T16:00:00Z",
  "started_at": "2025-10-22T16:05:00Z",
  "estimated_completion": "2025-10-23T14:30:00Z",
  "current_stage": "cascade_training",
  "progress": 45,
  "stages": [
    {
      "stage": "diversity_analysis",
      "status": "completed",
      "progress": 100,
      "started_at": "2025-10-22T16:05:00Z",
      "completed_at": "2025-10-22T20:00:00Z"
    },
    {
      "stage": "cascade_training",
      "status": "in_progress",
      "progress": 45,
      "started_at": "2025-10-22T20:01:00Z",
      "models_trained": 7,
      "models_total": 15
    },
    {
      "stage": "collapse_detection",
      "status": "pending",
      "progress": 0
    },
    {
      "stage": "report_generation",
      "status": "pending",
      "progress": 0
    }
  ]
}
```

Response (Completed):

```json
{
  "validation_id": "val_ghi789",
  "dataset_id": "ds_def456",
  "status": "completed",
  "created_at": "2025-10-22T16:00:00Z",
  "completed_at": "2025-10-23T14:30:00Z",
  "results": {
    "risk_score": 23,
    "risk_level": "low",
    "predicted_performance": {
      "accuracy": 0.87,
      "confidence_interval": [0.84, 0.90],
      "confidence_level": 0.95
    },
    "collapse_probability": 0.05,
    "dimensions": {
      "distribution_fidelity": 92,
      "correlation_preservation": 88,
      "diversity_retention": 85,
      "rare_pattern_handling": 78,
      "temporal_stability": 91,
      "semantic_coherence": 89
    },
    "recommendation": "approved",
    "warranty_eligible": true
  },
  "report_url": "https://api.synthos.ai/v1/validations/val_ghi789/report",
  "certificate_url": "https://api.synthos.ai/v1/validations/val_ghi789/certificate"
}
```

GET /validations

Purpose: List all validation jobs

Query Parameters:

- `page`, `page_size`, `status`, `sort_by` (same as datasets)

Response:

```json
```

```
{
  "validations": [
    {
      "validation_id": "val_ghi789",
      "dataset_id": "ds_def456",
      "dataset_name": "training_data.csv",
      "status": "completed",
      "risk_score": 23,
      "created_at": "2025-10-22T16:00:00Z",
      "completed_at": "2025-10-23T14:30:00Z"
    }
  ],
  "pagination": {...}
}
```

GET /validations/{validation_id}/report

Purpose: Download detailed validation report (PDF)

Response: Binary PDF file with headers:

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="validation_val_ghi789.pdf"
```

GET /validations/{validation_id}/certificate

Purpose: Download validation certificate (if passed)

Response: Binary PDF file with digital signature

2.4 Collapse Detection & Recommendations

GET /validations/{validation_id}/collapse-details

Purpose: Get detailed collapse analysis (if detected)

Response:

```
json
```

```
{
  "validation_id": "val_ghi789",
  "collapse_detected": true,
  "collapse_type": "Type B: Correlation Collapse",
  "severity": "medium",
  "affected_dimensions": [
    {
      "dimension": "correlation_preservation",
      "score": 62,
      "threshold": 70,
      "impact": "high"
    },
    {
      "dimension": "rare_pattern_handling",
      "score": 58,
      "threshold": 70,
      "impact": "medium"
    }
  ],
  "problematic_regions": [
    {
      "region_id": "reg_001",
      "row_range": [1200000, 1500000],
      "issue": "duplicate_entities",
      "impact_score": 35,
      "affected_columns": ["user_id", "email"]
    },
    {
      "region_id": "reg_002",
      "row_range": [3000000, 3200000],
      "issue": "outlier_density",
      "impact_score": 28,
      "affected_columns": ["age", "income"]
    }
  ],
  "root_causes": [
    {
      "cause": "Duplicate entities detected",
      "percentage": 60,
      "description": "~15% of rows are duplicates (300K rows)"
    },
    {
      "cause": "Excessive outliers",
      "percentage": 30,
      "description": "Outlier density >10% in numerical columns"
    },
```

```
    {
      "cause": "Correlation inconsistency",
      "percentage": 10,
      "description": "Age-income correlation varies wildly across subsets"
    }
  ]
}
```

GET /validations/{validation_id}/recommendations

Purpose: Get actionable recommendations to fix data

Response:

```
json
```

```
{
  "validation_id": "val_ghi789",
  "recommendations": [
    {
      "priority": 1,
      "category": "data_removal",
      "title": "Remove duplicate entities",
      "description": "Remove rows 1.2M-1.5M (duplicate user accounts)",
      "impact": {
        "current_risk_score": 62,
        "expected_risk_score": 38,
        "improvement": 24
      },
      "implementation": {
        "method": "deduplication",
        "affected_rows": 300000,
        "estimated_time": "2 hours"
      }
    },
    {
      "priority": 2,
      "category": "data_smoothing",
      "title": "Normalize outliers",
      "description": "Cap outliers at 3σ in age and income columns",
      "impact": {
        "current_risk_score": 38,
        "expected_risk_score": 22,
        "improvement": 16
      },
      "implementation": {
        "method": "winsorization",
        "affected_rows": 50000,
        "estimated_time": "30 minutes"
      }
    },
    {
      "priority": 3,
      "category": "data_augmentation",
      "title": "Oversample rare patterns",
      "description": "Increase representation of minority classes by 2x",
      "impact": {
        "current_risk_score": 22,
        "expected_risk_score": 15,
        "improvement": 7
      },
      "implementation": {
```

```
      "method": "oversampling",
      "affected_rows": 100000,
      "estimated_time": "1 hour"
    }
  }
],
"combined_impact": {
  "current_risk score": 62,
  "expected_risk_score": 15,
  "total_improvement": 47,
  "estimated_time": "3.5 hours"
}
}
```

2.5 Warranty & Certification

POST /validations/{validation_id}/warranty/request

Purpose: Request performance warranty for validation

Request:

```
{
  "validation_id": "val_ghi789",
  "training_details": {
    "model_size": "1B",
    "architecture": "transformer",
    "expected_start_date": "2025-11-01",
    "estimated_compute_cost": 75000000
  }
}
```

Response:

```
```

```
{
  "warranty_id": "war_jkl012",
  "validation_id": "val_ghi789",
  "status": "pending_review",
  "eligibility": {
    "risk_score": 23,
    "threshold": 25,
    "eligible": true
  },
  "terms": {
    "coverage_type": "performance_accuracy",
    "max_payout": 70000,
    "deductible": 10000,
    "premium": 10500,
    "duration_days": 90
  },
  "conditions": [
    "Customer must follow all recommendations",
    "Training must start within 60 days",
    "Customer must use specified model architecture",
    "Performance deviation must exceed 20%"
  ],
  "review_estimated": "2025-10-24T12:00:00Z"
}
```

GET /warranties/{warranty_id}

Purpose: Get warranty status and details

Response:

```
json
```

```
{
  "warranty_id": "war_jkl012",
  "validation_id": "val_ghi789",
  "status": "active",
  "purchased_at": "2025-10-24T14:00:00Z",
  "expires_at": "2026-01-22T14:00:00Z",
  "terms": {...},
  "customer_obligations": [
    {
      "obligation": "Notify before training",
      "status": "completed",
      "completed_at": "2025-10-30T10:00:00Z"
    },
    {
      "obligation": "Follow recommendations",
      "status": "verified",
      "verified_at": "2025-10-30T12:00:00Z"
    }
  ]
}
```

POST /warranties/{warranty_id}/claim

Purpose: File warranty claim

Request:

```
json

{
  "warranty_id": "war_jkl012",
  "claim_reason": "performance_deviation",
  "training_results": {
    "actual_accuracy": 0.65,
    "predicted_accuracy": 0.87,
    "deviation": 0.22
  },
  "supporting_documents": [
    "training_logs.txt",
    "evaluation_results.json"
  ]
}
```

Response:

```json
{
  "claim_id": "clm_mno345",
  "warranty_id": "war_jkl012",
  "status": "under_review",
  "filed_at": "2025-11-15T10:00:00Z",
  "estimated_resolution": "2025-11-29T10:00:00Z",
  "next_steps": [
    "Upload training logs and evaluation results",
    "Technical review (3-5 days)",
    "Legal approval (1-2 days)",
    "Payout processing (1-2 days)"
  ]
}
```

2.6 Monitoring & Analytics

GET /analytics/usage

Purpose: Get customer usage statistics

Response:

```json
{
  "period": "2025-10",
  "datasets_uploaded": 12,
  "validations_completed": 8,
  "total_rows_validated": 4500000000,
  "average_risk_score": 28,
  "warranty_contracts": 5,
  "subscription_tier": "professional",
  "usage_limits": {
    "validations_limit": 20,
    "validations_used": 8,
    "validations_remaining": 12
  }
}
```

GET /analytics/validation-history

Purpose: Get historical validation results

Query Parameters:

- `start_date`, `end_date`, `risk_level`

Response:

```json
{
  "validations": [
    {
      "validation_id": "val_ghi789",
      "dataset_name": "training_data.csv",
      "risk_score": 23,
      "completed_at": "2025-10-23T14:30:00Z"
    }
  ],
  "trends": {
    "average_risk_score": 28,
    "average_risk_score_change": -5,
    "total_compute_saved": 150000000
  }
}
```

III. INTERNAL gRPC SERVICE APIs

3.1 Job Orchestrator ↔ Validation Engine

Proto Definition:

```protobuf
```

```
service ValidationEngine {
  // Phase 1: Data profiling
  rpc ProfileDataset(ProfileRequest) returns (ProfileResponse);

  // Phase 2: Diversity analysis
  rpc AnalyzeDiversity(DiversityRequest) returns (DiversityResponse);

  // Phase 3: Pre-training risk assessment
  rpc PreScreenRisk(PreScreenRequest) returns (PreScreenResponse);

  // Phase 4: Multi-scale cascade training
  rpc TrainCascade(CascadeRequest) returns (stream CascadeProgress);

  // Phase 5: Get final predictions
  rpc GetPredictions(PredictionRequest) returns (PredictionResponse);
}
```

ProfileDataset

Purpose: Initial data profiling and schema detection

Request:

```protobuf
message ProfileRequest {
  string dataset_id = 1;
  string s3_path = 2;
  int64 row_count = 3;
}
```

Response:

```protobuf
```

```
message ProfileResponse {
  string dataset_id = 1;
  repeated ColumnProfile columns = 2;
  DataQualityMetrics quality = 3;
  bool has_pii = 4;
}

message ColumnProfile {
  string name = 1;
  string data_type = 2;
  double null_rate = 3;
  double unique_rate = 4;
  StatisticalProfile stats = 5;
}
```

AnalyzeDiversity

Purpose: Perform stratified sampling and diversity analysis

Request:

```protobuf
message DiversityRequest {
  string dataset_id = 1;
  string s3_path = 2;
  int32 sample_size = 3; // e.g., 20000000
}
```

Response:

```protobuf
```

```
message DiversityResponse {
  string dataset_id = 1;
  string sample_s3_path = 2;
  DiversityMetrics metrics = 3;
  repeated Cluster clusters = 4;
  SamplingConfidence confidence = 5;
}

message DiversityMetrics {
  double entropy = 1;
  double gini_coefficient = 2;
  int32 cluster_count = 3;
  repeated CorrelationMatrix correlations = 4;
}
```

PreScreenRisk

Purpose: Match against collapse signature library

Request:

```protobuf
message PreScreenRequest {
  string dataset_id = 1;
  DiversityMetrics diversity = 2;
}
```

Response:

```protobuf
```

```protobuf
message PreScreenResponse {
  string dataset_id = 1;
  int32 pre_risk_score = 2; // 0-100
  repeated SignatureMatch matches = 3;
  bool should_proceed = 4;
  string recommendation = 5;
}

message SignatureMatch {
  string signature_id = 1;
  string collapse_type = 2;
  double similarity = 3;
  string historical_outcome = 4;
}
```

TrainCascade

Purpose: Train multi-scale cascade models (streaming response)

Request:

```protobuf
message CascadeRequest {
  string dataset_id = 1;
  string sample_s3_path = 2;
  CascadeConfig config = 3;
}

message CascadeConfig {
  repeated ModelTier tiers = 1;
  string target_architecture = 2;
  int64 target_model_size = 3;
}

message ModelTier {
  int32 tier_number = 1;
  int64 model_size = 2;
  int32 num_variants = 3;
  int32 training_rows = 4;
}
```

Response (Stream):

```protobuf
```

```
message CascadeProgress {
  string dataset_id = 1;
  int32 current_tier = 2;
  int32 current_variant = 3;
  int32 models_completed = 4;
  int32 models_total = 5;
  double progress_percent = 6;

  // When model completes
  optional ModelResult result = 7;
}

message ModelResult {
  int32 tier = 1;
  int32 variant = 2;
  int64 model_size = 3;
  TrainingMetrics metrics = 4;
  bool collapse_detected = 5;
}
```

GetPredictions

Purpose: Get final predictions and extrapolations

Request:

```protobuf
message PredictionRequest {
  string dataset_id = 1;
  repeated ModelResult cascade_results = 2;
  int64 target_model_size = 3;
}
```

Response:

```protobuf
```

```
message PredictionResponse {
  string dataset_id = 1;
  double predicted_accuracy = 2;
  ConfidenceInterval confidence = 3;
  ScalingLawCoefficients scaling = 4;
  int32 final_risk_score = 5;
}

message ConfidenceInterval {
  double lower_bound = 1;
  double upper_bound = 2;
  double confidence_level = 3;
}
```

3.2 Validation Engine ↔ Collapse Engine

Proto Definition:

```protobuf
service CollapseEngine {
  // Detect collapse in cascade results
  rpc DetectCollapse(CollapseRequest) returns (CollapseResponse);

  // Locate problematic data regions
  rpc LocalizeProblems(LocalizationRequest) returns (LocalizationResponse);

  // Generate recommendations
  rpc GenerateRecommendations(RecommendationRequest) returns (RecommendationResponse);

  // Validate fixes (re-validation)
  rpc ValidateFixes(FixValidationRequest) returns (FixValidationResponse);
}
```

DetectCollapse

Purpose: Analyze cascade results for collapse signals

Request:

```protobuf
```

```
message CollapseRequest {
  string dataset_id = 1;
  repeated ModelResult cascade_results = 2;
  DiversityMetrics original_diversity = 3;
}
```

Response:

protobuf

```
message CollapseResponse {
  string dataset_id = 1;
  bool collapse_detected = 2;
  string collapse_type = 3; // "Type A", "Type B", etc.
  string severity = 4; // "low", "medium", "high"

  repeated DimensionScore dimensions = 5;
  repeated RootCause root_causes = 6;
}

message DimensionScore {
  string dimension = 1; // "distribution_fidelity", etc.
  int32 score = 2; // 0-100
  int32 threshold = 3;
  bool passed = 4;
}

message RootCause {
  string cause = 1;
  double percentage = 2;
  string description = 3;
}
```

LocalizeProblems

Purpose: Pinpoint exact rows causing collapse

Request:

protobuf

```
message LocalizationRequest {
  string dataset_id = 1;
  string sample_s3_path = 2;
  repeated ModelResult cascade_results = 3;
  CollapseResponse collapse_info = 4;
}
```

Response:

```protobuf
message LocalizationResponse {
  string dataset_id = 1;
  repeated ProblematicRegion regions = 2;
  GradientHeatmap heatmap = 3;
  repeated AblationResult ablations = 4;
}

message ProblematicRegion {
  string region_id = 1;
  int64 row_start = 2;
  int64 row_end = 3;
  string issue_type = 4; // "duplicates", "outliers", etc.
  double impact_score = 5;
  repeated string affected_columns = 6;
}

message AblationResult {
  string region_id = 1;
  int32 risk_score_before = 2;
  int32 risk_score_after = 3;
  int32 improvement = 4;
  bool hypothesis_confirmed = 5;
}
```

GenerateRecommendations

Purpose: Create actionable fix recommendations

Request:

```protobuf
```

```
message RecommendationRequest {
  string dataset_id = 1;
  LocalizationResponse localization = 2;
  CollapseResponse collapse_info = 3;
}
```

Response:

```protobuf
message RecommendationResponse {
  string dataset_id = 1;
  repeated Recommendation recommendations = 2;
  CombinedImpact combined_impact = 3;
}

message Recommendation {
  int32 priority = 1;
  string category = 2; // "data_removal", "data_smoothing", etc.
  string title = 3;
  string description = 4;
  Impact impact = 5;
  Implementation implementation = 6;
}

message Impact {
  int32 current_risk_score = 1;
  int32 expected_risk_score = 2;
  int32 improvement = 3;
}

message Implementation {
  string method = 1;
  int64 affected_rows = 2;
  string estimated_time = 3;
}

message CombinedImpact {
  int32 current_risk_score = 1;
  int32 expected_risk_score = 2;
  int32 total_improvement = 3;
  string estimated_time = 4;
}
```

3.3 Job Orchestrator ↔ Data Service

Proto Definition:

```protobuf
service DataService {
  // Get signed upload URL
  rpc GetUploadURL(UploadRequest) returns (UploadResponse);

  // Mark upload complete
  rpc CompleteUpload(CompleteRequest) returns (CompleteResponse);

  // Get dataset metadata
  rpc GetDatasetMetadata(MetadataRequest) returns (MetadataResponse);

  // Stream dataset chunks
  rpc StreamDataset(StreamRequest) returns (stream DataChunk);
}
```

GetUploadURL

Request:

```protobuf
message UploadRequest {
  string user_id = 1;
  string filename = 2;
  int64 file_size = 3;
  string file_type = 4;
}
```

Response:

```protobuf
message UploadResponse {
  string dataset_id = 1;
  string upload_url = 2;
  string upload_method = 3;
  int32 chunk_size = 4;
  int32 expires_in = 5;
}
```

StreamDataset

Purpose: Stream dataset in chunks for processing

Request:

```protobuf
message StreamRequest {
  string dataset_id = 1;
  int64 start_row = 2;
  int64 end_row = 3;
  int32 chunk_size = 4;
}
```

Response (Stream):

```protobuf
message DataChunk {
  string dataset_id = 1;
  int64 chunk_number = 2;
  int64 start_row = 3;
  int64 end_row = 4;
  bytes data = 5; // Compressed data (Parquet or CSV)
  string format = 6;
}
```

3.4 Collapse Engine ↔ Signature Library Service

Proto Definition:

```protobuf
service SignatureLibrary {
  // Match dataset fingerprint against library
  rpc MatchSignatures(MatchRequest) returns (MatchResponse);

  // Add new collapse signature
  rpc AddSignature(AddSignatureRequest) returns (AddSignatureResponse);

  // Update signature with outcome
  rpc UpdateOutcome(UpdateOutcomeRequest) returns (UpdateOutcomeResponse);
}
```

MatchSignatures

Request:

```protobuf
message MatchRequest {
  string dataset_id = 1;
  DatasetFingerprint fingerprint = 2;
}

message DatasetFingerprint {
  repeated double distribution_shape = 1;
  repeated double entropy_measures = 2;
  repeated double correlation_values = 3;
  repeated double temporal_features = 4;
}
```

Response:

```protobuf
message MatchResponse {
  repeated SignatureMatch matches = 1;
  int32 pre_risk_score = 2;
  string recommendation = 3;
}

message SignatureMatch {
  string signature_id = 1;
  string collapse_type = 2;
  double similarity = 3;
  string historical_outcome = 4;
  string dataset_source = 5;
}
```

AddSignature

Purpose: Add new collapse pattern to library

Request:

```protobuf
```

```
message AddSignatureRequest {
  string validation_id = 1;
  DatasetFingerprint fingerprint = 2;
  string collapse_type = 3;
  int32 risk_score = 4;
  string root_cause = 5;
}
```

Response:

```protobuf
message AddSignatureResponse {
  string signature_id = 1;
  bool added = 2;
  int32 total_signatures = 3;
}
```

IV. REDESIGNED FRONTEND ARCHITECTURE

4.1 Application Structure

```
synthos-frontend/
├── src/
│   ├── pages/
│   │   ├── auth/
│   │   │   ├── login.tsx
│   │   │   ├── register.tsx
│   │   │   └── forgot-password.tsx
│   │   ├── dashboard/
│   │   │   └── index.tsx
│   │   ├── datasets/
│   │   │   ├── index.tsx
│   │   │   ├── upload.tsx
│   │   │   └── [id].tsx
│   │   ├── validations/
│   │   │   ├── index.tsx
│   │   │   ├── create.tsx
│   │   │   ├── [id]/
│   │   │   │   ├── index.tsx
│   │   │   │   ├── live.tsx
│   │   │   │   ├── results.tsx
│   │   │   │   ├── collapse-analysis.tsx
│   │   │   │   └── recommendations.tsx
```

```
│   │   ├── warranties/
│   │   │   ├── index.tsx
│   │   │   ├── request.tsx
│   │   │   └── [id].tsx
│   │   ├── analytics/
│   │   │   ├── usage.tsx
│   │   │   └── history.tsx
│   │   ├── settings/
│   │   │   ├── profile.tsx
│   │   │   ├── billing.tsx
│   │   │   └── api-keys.tsx
│   │   └── admin/ (enterprise tier only)
│   │       ├── users.tsx
│   │       └── team.tsx
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Navbar.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   └── Footer.tsx
│   │   ├── datasets/
│   │   │   ├── DatasetCard.tsx
│   │   │   ├── DatasetTable.tsx
│   │   │   ├── UploadZone.tsx
│   │   │   └── SchemaViewer.tsx
│   │   ├── validations/
│   │   │   ├── ValidationCard.tsx
│   │   │   ├── ValidationStatusBadge.tsx
│   │   │   ├── RiskScoreGauge.tsx
│   │   │   ├── ProgressTracker.tsx
│   │   │   ├── LiveTrainingMonitor.tsx
│   │   │   ├── DimensionScoreCard.tsx
│   │   │   ├── CollapseHeatmap.tsx
│   │   │   └── RecommendationCard.tsx
│   │   ├── warranties/
│   │   │   ├── WarrantyCard.tsx
│   │   │   ├── EligibilityChecker.tsx
│   │   │   └── ClaimForm.tsx
│   │   ├── charts/
│   │   │   ├── ScalingLawChart.tsx
│   │   │   ├── DiversityChart.tsx
│   │   │   ├── CorrelationMatrix.tsx
│   │   │   └── TrendChart.tsx
│   │   └── common/
│   │       ├── Button.tsx
│   │       ├── Card.tsx
│   │       ├── Badge.tsx
│   │       ├── Modal.tsx
```

```
│   │       └── LoadingSpinner.tsx
│   ├── lib/
│   │   ├── api/
│   │   │   ├── client.ts
│   │   │   ├── auth.ts
│   │   │   ├── datasets.ts
│   │   │   ├── validations.ts
│   │   │   └── warranties.ts
│   │   ├── hooks/
│   │   │   ├── useAuth.ts
│   │   │   ├── useDatasets.ts
│   │   │   ├── useValidations.ts
│   │   │   ├── useWebSocket.ts
│   │   │   └── usePolling.ts
│   │   └── utils/
│   │       ├── formatters.ts
│   │       ├── validators.ts
│   │       └── constants.ts
│   └── types/
│       ├── dataset.ts
│       ├── validation.ts
│       └── warranty.ts
```

4.2 Key Frontend Pages (Detailed)

Page 1: Dashboard (`/dashboard`)

Purpose: Overview of all validations, datasets, and account status

Layout:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Header: Synthos | Dashboard        [Profile] [Logout]                        │
├──────────────────────────────────────────────────────────────────────────────┤
│ Sidebar │ Main Content                                                       │
│ ────────┼────────────────────────────────────────────────────────────────────│
│ Dashboard│ 📊 Overview                                                        │
│ Datasets │  ┌──────────┬──────────┬──────────┬──────────┐                   │
│ Validate │  │ Datasets │Valid Jobs│  Risk    │ Warranty │                   │
│ Warranty │  │    12    │    8     │ Score    │    5     │                   │
│ Analytics│  │          │          │   28     │ Active   │                   │
│ Settings │  └──────────┴──────────┴──────────┴──────────┘                   │
│          │                                                                   │
│          │ 📈 Recent Validations                                             │
│          │  ┌──────────────────────────────────────────────────────────┐    │
│          │  │ training_data.csv     ✓ 23   Oct 23, 2025                │    │
│          │  │ customer_synth.json   ⚠ 45   Oct 20, 2025                │    │
│          │  │ model_inputs.parquet  ⏳ --   In Progress                 │    │
│          │  └──────────────────────────────────────────────────────────┘    │
│          │                                                                   │
│          │ 📊 Risk Score Trend (Last 30 Days)                                │
│          │ [Line chart showing risk scores over time]                        │
│          │                                                                   │
│          │ 💰 Compute Saved: $150M                                           │
└──────────────────────────────────────────────────────────────────────────────┘
```

Key Components:

- **Stats Cards:** Total datasets, validations, avg risk score, active warranties
- **Recent Validations Table:** List recent jobs with status badges
- **Risk Score Trend Chart:** Line chart showing improvement over time
- **Compute Savings Counter:** Animated counter showing total savings

API Calls:

- `GET /analytics/usage`
- `GET /validations?page=1&page_size=5&sort_by=created_at`
- `GET /analytics/validation-history?days=30`

Page 2: Datasets List (`/datasets`)

Purpose: View and manage all uploaded datasets

Layout:

```
┌──────────────────────────────────────────────────────────────────────┐
│ Datasets                                   [+ Upload New]            │
├──────────────────────────────────────────────────────────────────────┤
│ Filters: [All Status ▼] [Sort: Date ▼]   🔍 Search...                │
├──────────────────────────────────────────────────────────────────────┤
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ 📄 training_data.csv                                    ✓    │   │
│  │ 500M rows | 50 columns | Uploaded Oct 22, 2025               │   │
│  │ Quality Score: 87/100 | 3 Validations                        │   │
│  │ [View Details] [Validate] [Download] [Delete]                │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ 📄 customer_synth.json                                  ⚠    │   │
│  │ 250M rows | 35 columns | Uploaded Oct 20, 2025               │   │
│  │ Quality Score: 62/100 | 1 Validation                         │   │
│  │ [View Details] [Re-validate] [Download]                      │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│ Pagination: [1] 2 3 ... 8                                            │
└──────────────────────────────────────────────────────────────────────┘
```

Key Features:

- **Upload Button:** Opens upload modal with drag-and-drop
- **Filter/Search:** Filter by status, search by filename
- **Dataset Cards:** Show key metrics, quality score, action buttons
- **Bulk Actions:** Select multiple datasets, bulk delete

API Calls:

- `GET /datasets?page=1&page_size=20`
- `DELETE /datasets/{id}` (on delete action)

Page 3: Dataset Upload (`/datasets/upload`)

Purpose: Upload new dataset with guided flow

Layout:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Upload Dataset                    Step 1 of 3    │
├──────────────────────────────────────────────────────────────────────────────┤
│                                  │
│       ┌──────────────────────────────────────────────┐          │
│       │                    │          │
│       │  📤 Drag and drop file here    │          │
│       │     or click to browse       │          │
│       │                    │          │
│       │ Supported: CSV, JSON, Parquet   │          │
│       │ Max size: 100GB              │          │
│       └──────────────────────────────────────────────┘          │
│                                  │
│ Dataset Details:                          │
│  ┌──────────────────────────────────────────────────────────┐          │
│  │ Description: [Optional description...]    │          │
│  └──────────────────────────────────────────────────────────┘          │
│                                  │
│ Privacy Settings:                         │
│ ☐ This dataset contains PII                    │
│ ☐ Apply strict data retention (30 days)              │
│                                  │
│                  [Cancel] [Next: Upload →]     │
└──────────────────────────────────────────────────────────────────────────────┘
```

Upload Flow:

Step 1: File Selection

- Drag-and-drop zone
- File type validation
- Size check
- Optional metadata

Step 2: Upload Progress

```
┌──────────────────────────────────────────────────────────────┐
│ Uploading: training_data.csv          Step 2 of 3            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│ ▓▓▓
```

Page 4: Validation Creation (`/validations/create`)

Purpose: Configure and start new validation job

Layout:

```
┌──────────────────────────────────────────────────────────────────────────┐
│ Create Validation Job                                                    │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│ 1. Select Dataset                                                        │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │ ◉ training_data.csv (500M rows)                                  │    │
│  │ ○ customer_synth.json (250M rows)                                │    │
│  │ ○ model_inputs.parquet (1B rows)                                 │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
│ 2. Validation Settings                                                   │
│ Target Model:                                                            │
│ Architecture: [Transformer ▼]                                            │
│ Model Size: [1 Billion parameters ▼]                                     │
│                                                                          │
│ Priority: ○ Standard (48 hours)  ◉ Express (24 hours +50%)               │
│                                                                          │
│ 3. Additional Options                                                    │
│ ☑ Enable performance warranty eligibility check                          │
│ ☐ Generate detailed collapse analysis                                    │
│ ☑ Email notification on completion                                       │
│                                                                          │
│ ──────────────────────────────────────────────────────────────────────── │
│ Estimated Cost: $35,000                                                  │
│ Estimated Completion: Oct 24, 2025 2:30 PM                               │
│                                                                          │
│                    [Cancel] [Start Validation →]                         │
└──────────────────────────────────────────────────────────────────────────┘
```

Key Features:

- **Dataset Selection:** Radio buttons with dataset preview
- **Model Configuration:** Target architecture and size
- **Priority Options:** Standard vs. Express (cost difference)
- **Warranty Toggle:** Enable warranty eligibility check
- **Cost Estimator:** Real-time cost calculation based on options

API Calls:

- `GET /datasets?status=processed` (get available datasets)
- `POST /validations/create` (start validation)

Page 5: Live Validation Monitor (`/validations/{id}/live`)

Purpose: Real-time monitoring of validation progress

Layout:

```
┌─────────────────────────────────────────────────────────────────────────┐
│ Validation: val_ghi789                    ⏳ In Progress                │
│ Dataset: training_data.csv (500M rows)                                  │
├─────────────────────────────────────────────────────────────────────────┤
│                                                                         │
│ Overall Progress: ▓▓▓
```

- **Auto-Refresh:** WebSocket or polling every 10 seconds

API Calls:

- `GET /validations/{id}` (poll every 10s)
- WebSocket: `ws://api.synthos.ai/v1/validations/{id}/stream`

Page 6: Validation Results (`/validations/{id}/results`)

Purpose: View completed validation results

Layout:

```
┌──────────────────────────────────────────────────────────────────────────┐
│ Validation Results: val_ghi789          ✓ Completed                      │
│ Dataset: training_data.csv | Completed: Oct 23, 2:30 PM                  │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │        RISK SCORE: 23/100                                        │    │
│  │  [========▒▒▒
```

API Calls:

- `GET /validations/{id}` (get final results)
- `GET /validations/{id}/report` (download PDF)

Page 7: Collapse Analysis (`/validations/{id}/collapse-analysis`)

Purpose: Deep dive into detected collapse (if any)

Layout:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Collapse Analysis: val_xyz456          ⚠ Issues Found │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ Risk Score: 62/100 (MEDIUM RISK)                                             │
│ Collapse Type: Type B - Correlation Collapse                                 │
│ Severity: Medium                                                             │
│                                                                              │
│ 🔍 Affected Dimensions:                                                      │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │ ⚠ Correlation Preservation: 62/100 (Threshold: 70)                   │   │
│  │   Impact: High - Relationships between features                      │   │
│  │   degraded significantly                                             │   │
│  │                                                                      │   │
│  │ ⚠ Rare Pattern Handling: 58/100 (Threshold: 70)                      │   │
│  │   Impact: Medium - Minority classes underrepresent                   │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│ 🗺 Problematic Data Regions:                                                 │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │ Region 1: Rows 1.2M - 1.5M                                           │   │
│  │ Issue: Duplicate Entities                                            │   │
│  │ Impact Score: 35/100                                                 │   │
│  │ Affected Columns: user_id, email                                     │   │
│  │ [View Heatmap] [Download Sample]                                     │   │
│  ├──────────────────────────────────────────────────────────────────────┤   │
│  │ Region 2: Rows 3.0M - 3.2M                                           │   │
│  │ Issue: Outlier Density (>10%)                                        │   │
│  │ Impact Score: 28/100                                                 │   │
│  │ Affected Columns: age, income                                        │   │
│  │ [View Distribution] [Download Sample]                                │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│ 📊 Root Cause Breakdown:                                                     │
│ [Pie chart: 60% Duplicates | 30% Outliers | 10% Other]                       │
│                                                                              │
│ [View Recommendations →] [Download Full Analysis]                            │
└──────────────────────────────────────────────────────────────────────────────┘
```

Key Features:

- **Collapse Type Badge:** Visual indicator of collapse category
- **Affected Dimensions:** Failed dimensions with explanations
- **Problematic Regions Table:** Specific row ranges with issues

- **Root Cause Pie Chart:** Visual breakdown of causes
- **Interactive Heatmaps:** Click to zoom into problem areas

API Calls:

- `GET /validations/{id}/collapse-details`

Page 8: Recommendations (`/validations/{id}/recommendations`)

Purpose: Actionable steps to fix data and improve risk score

Layout:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Recommendations: val_xyz456                    62 → 15                       │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ By implementing these fixes, your risk score will improve                    │
│ from 62 (MEDIUM RISK) to 15 (LOW RISK) - a 47-point drop!                    │
│                                                                              │
│ ──────────────────────────────────────────────────────────────────────────── │
│                                                                              │
│ ⚠ PRIORITY 1 (Critical)                                                      │
│ ┌────────────────────────────────────────────────────────────────────────┐   │
│ │ Remove Duplicate Entities                                              │   │
│ │                                                                        │   │
│ │ Description: Remove rows 1.2M-1.5M containing                          │   │
│ │ duplicate user accounts (~15% of dataset)                              │   │
│ │                                                                        │   │
│ │ Impact: 62 → 38 (24-point improvement)                                 │   │
│ │ Affected Rows: 300,000 rows                                            │   │
│ │ Implementation: Deduplication script                                   │   │
│ │ Estimated Time: 2 hours                                                │   │
│ │                                                                        │   │
│ │ [Download Script] [View Sample Data] [Mark as Done]                    │   │
│ └────────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│ ⚠ PRIORITY 2 (High)                                                          │
│ ┌────────────────────────────────────────────────────────────────────────┐   │
│ │ Normalize Outliers in Numerical Columns                                │   │
│ │                                                                        │   │
│ │ Description: Cap outliers at 3σ in age and income                      │   │
│ │ columns to prevent gradient instability                                │   │
│ │                                                                        │   │
│ │ Impact: 38 → 22 (16-point improvement)                                 │   │
│ │ Affected Rows: 50,000 rows                                             │   │
│ │ Implementation: Winsorization                                          │   │
│ │ Estimated Time: 30 minutes                                             │   │
│ │                                                                        │   │
│ │ [Download Script] [View Distributions] [Mark as Done]                  │   │
│ └────────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│ ℹ PRIORITY 3 (Medium)                                                        │
│ ┌────────────────────────────────────────────────────────────────────────┐   │
│ │ Oversample Rare Patterns                                               │   │
│ │                                                                        │   │
│ │ Description: Increase minority class representation                    │   │
│ │ by 2x to improve model robustness on edge cases                        │   │
│ │                                                                        │   │
```

```
│  │ Impact: 22 → 15 (7-point improvement)          │  │
│  │ Affected Rows: +100,000 rows (synthetic generation) │  │
│  │ Implementation: SMOTE oversampling             │  │
│  │ Estimated Time: 1 hour                         │  │
│  │                                                │  │
│  │ [Download Script] [View Strategy] [Mark as Done]  │  │
│  └──────────────────────────────────────────────────┘  │
│                                                        │
│ ──────────────────────────────────────────────────────── │
│                                                        │
│ Combined Impact:                                       │
│ • Current Risk Score: 62/100 (MEDIUM RISK)             │
│ • Expected Risk Score: 15/100 (LOW RISK)               │
│ • Total Improvement: 47 points                         │
│ • Total Time: 3.5 hours                                │
│                                                        │
│ [Download All Scripts] [Request Re-validation]         │
└────────────────────────────────────────────────────────┘
```

Key Features:

- **Priority-Ordered Cards:** Visual hierarchy (color-coded)
- **Impact Visualization:** Before/after risk scores
- **Implementation Details:** Specific methods and scripts
- **Action Buttons:** Download scripts, mark complete, request help
- **Combined Impact Summary:** Total improvement if all implemented

API Calls:

- `GET /validations/{id}/recommendations`
- Download scripts (generated Python/SQL code for fixes)

Page 9: Warranty Request (`/warranties/request`)

Purpose: Request performance warranty for validated dataset

Layout:

Request Performance Warranty

1. Select Validation

◉ val_ghi789 - training_data.csv
Risk Score: 23/100 ✓ | Completed: Oct 23, 2025
Eligible for Warranty ✅

○ val_xyz456 - customer_synth.json
Risk Score: 62/100 ⚠ | Completed: Oct 20, 2025
NOT Eligible (Risk > 25) ❌

2. Training Details

Model Architecture: [Transformer (GPT-style) ▼]
Model Size: [1 Billion parameters ▼]
Expected Start Date: [Nov 1, 2025 📅]
Estimated Compute Cost: [$75,000,000]

3. Warranty Terms

Coverage Type: Performance Accuracy Guarantee
Maximum Payout: $70,000 (2x validation fee)
Customer Deductible: $10,000
Coverage Threshold: >20% performance deviation
Premium: $10,500 (30% of validation fee)
Duration: 90 days from training start

4. Conditions & Requirements

You must:
☑ Follow all validation recommendations
☑ Use specified model architecture (Transformer)
☑ Start training within 60 days
☑ Notify us 7 days before training begins
☑ Provide training logs upon completion

Total Cost: $10,500
Review Period: 1-2 business days

☐ I have read and agree to the warranty terms

```
│                [Cancel] [Request Warranty →]          │
└──────────────────────────────────────────────────────────┘
```

Key Features:

- **Eligibility Check:** Visual indicators for eligible validations
- **Terms Preview:** Clear display of coverage, limits, premium
- **Conditions Checklist:** Customer obligations listed
- **Cost Calculator:** Real-time premium calculation
- **Legal Agreement:** Terms acceptance checkbox

API Calls:

- `GET /validations?warranty_eligible=true`
- `POST /validations/{id}/warranty/request`

Page 10: Warranty Dashboard (`/warranties`)

Purpose: Manage active warranties and claims

Layout:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Warranties & Coverage                    5 Active        │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ Active Warranties:                                                           │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │ 🛡️ war_jkl012 - training_data.csv          ✓ Active │   │
│  │ Coverage: $70,000 | Expires: Jan 22, 2026            │   │
│  │ Status: Training Notified ✓                          │   │
│  │                                                      │   │
│  │ Customer Obligations:                                │   │
│  │ ✓ Notify before training (Completed Oct 30)          │   │
│  │ ✓ Follow recommendations (Verified Oct 30)           │   │
│  │ ⏳ Provide training logs (Pending)                   │   │
│  │                                                      │   │
│  │ [View Details] [File Claim] [Download Certificate]   │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │ 🛡️ war_pqr345 - model_inputs.parquet      ⚠ Expiring │   │
│  │ Coverage: $100,000 | Expires: Nov 5, 2025 (12 days)  │   │
│  │ Status: Not Started ⚠                                │   │
│  │                                                      │   │
│  │ Action Required: Start training or request extension │   │
│  │                                                      │   │
│  │ [View Details] [Request Extension]                   │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│ Claims History:                                                              │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │ No claims filed ✓                                    │   │
│  │ Your validation accuracy: 100% (0 claims needed)     │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│ Total Coverage: $450,000                                                     │
│ Total Premium Paid: $67,500                                                  │
└──────────────────────────────────────────────────────────────────────────────┘
```

Key Features:

- **Warranty Cards:** Status badges, expiry warnings, obligations tracker
- **Claims Section:** File new claim, view claim history
- **Alerts:** Expiring warranties, action-required notices
- **Summary Stats:** Total coverage, premium paid, claim rate

API Calls:

- `GET /warranties`
- `GET /warranties/{id}`
- `POST /warranties/{id}/claim`

Page 11: Analytics & Usage (`/analytics/usage`)

Purpose: Detailed usage statistics and trends

Layout:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Analytics & Usage                    October 2025                            │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ 📊 Usage Summary:                                                            │
│ ┌──────────┬──────────┬──────────┬──────────────────────────────┐           │
│ │ Datasets │Validation│Total Rows│ Compute Saved                │           │
│ │   12     │    8     │   4.5B   │    $150M                     │           │
│ └──────────┴──────────┴──────────┴──────────────────────────────┘           │
│                                                                              │
│ 📈 Risk Score Trend (Last 6 Months):                                         │
│ [Line chart showing improvement: 45 → 38 → 32 → 28]                         │
│                                                                              │
│ 📊 Validation Breakdown by Risk Level:                                       │
│ [Bar chart: Low (5) | Medium (2) | High (1)]                                 │
│                                                                              │
│ 💰 Cost Analysis:                                                            │
│ ┌──────────────────────────────────────────────────────────────────────┐    │
│ │ Validation Spend:    $280,000                                        │    │
│ │ Warranty Premium:    $67,500                                         │    │
│ │ Total Investment:    $347,500                                        │    │
│ │                                                                      │    │
│ │ Compute Saved:       $150,000,000                                    │    │
│ │ ROI:                 43,100%                                         │    │
│ └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
│ 📅 Subscription Usage:                                                       │
│ Tier: Professional ($599/month)                                              │
│ Validations: 8 / 20 used (40%)                                               │
│ Resets: Nov 1, 2025 (8 days)                                                 │
│                                                                              │
│ [Export Report] [Upgrade Plan]                                               │
└──────────────────────────────────────────────────────────────────────────────┘
```

Key Features:

- **Usage Cards:** Key metrics at a glance
- **Trend Charts:** Risk score improvement, validation volumes
- **Cost/Benefit Analysis:** ROI calculation showing value
- **Subscription Status:** Usage limits, renewal date
- **Export Options:** Download usage reports (CSV, PDF)

API Calls:

- `GET /analytics/usage?period=2025-10`
- `GET /analytics/validation-history?months=6`

V. REAL-TIME COMMUNICATION PATTERNS

5.1 WebSocket Implementation

Connection Flow:

1. **Client Connects:**

```javascript
const ws = new WebSocket('wss://api.synthos.ai/v1/ws');
ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'authenticate',
    token: 'Bearer eyJhbGciOiJIUzI1NiIs...'
  }));
};
```

2. **Subscribe to Validation Updates:**

```javascript
ws.send(JSON.stringify({
  type: 'subscribe',
  channel: 'validation',
  validation_id: 'val_ghi789'
}));
```

3. **Receive Real-Time Updates:**

```javascript
```

```
ws.onmessage = (event) => {
  const update = JSON.parse(event.data);

  // Update types:
  // - progress: Overall progress update
  // - stage_complete: Stage finished
  // - model_complete: Individual model trained
  // - metrics: Live training metrics
  // - complete: Validation finished

  if (update.type === 'progress') {
    updateProgressBar(update.progress);
    updateStageStatus(update.current_stage);
  }

  if (update.type === 'model_complete') {
    addModelResult(update.model_result);
  }

  if (update.type === 'complete') {
    showResults(update.results);
  }
};
```

Message Formats:

Progress Update:

```
json

{
  "type": "progress",
  "validation_id": "val_ghi789",
  "progress": 45,
  "current_stage": "cascade_training",
  "models_completed": 7,
  "models_total": 15,
  "timestamp": "2025-10-23T08:30:15Z"
}
```

Stage Complete:

```
json
```

```
{
  "type": "stage_complete",
  "validation_id": "val_ghi789",
  "stage": "cascade_training",
  "duration_seconds": 108000,
  "next_stage": "collapse_detection",
  "timestamp": "2025-10-23T14:00:00Z"
}
```

Model Complete:

```json
{
  "type": "model_complete",
  "validation_id": "val_ghi789",
  "model": {
    "tier": 2,
    "variant": 3,
    "model_size": 10000000,
    "training_loss": 0.347,
    "validation_loss": 0.412,
    "collapse_detected": false
  },
  "timestamp": "2025-10-23T10:15:30Z"
}
```

5.2 Polling Fallback

For clients without WebSocket support:

```javascript
// Poll every 10 seconds
const pollInterval = setInterval(async () => {
  const response = await fetch(`/api/v1/validations/${validationId}`);
  const data = await response.json();

  updateUI(data);

  if (data.status === 'completed' || data.status === 'failed') {
    clearInterval(pollInterval);
  }
}, 10000);
```

VI. DATA FLOW EXAMPLES

6.1 Complete Validation Flow

Step-by-Step Data Flow:

1. **Customer uploads dataset** → Frontend
2. **Frontend requests signed URL** → `POST /datasets/upload` → API Gateway
3. **API Gateway calls Data Service** → `GetUploadURL` (gRPC)
4. **Data Service generates S3 signed URL** → Returns to Frontend
5. **Frontend uploads directly to S3** → Chunked upload
6. **Frontend marks upload complete** → `POST /datasets/{id}/complete` → API Gateway
7. **API Gateway triggers Job Orchestrator** → `CreateProcessingJob` (gRPC)
8. **Job Orchestrator calls Validation Engine** → `ProfileDataset` (gRPC)
9. **Validation Engine processes data** → Streams from S3, analyzes, returns profile
10. **Job Orchestrator updates database** → Dataset status = "processed"
11. **Frontend polls for status** → `GET /datasets/{id}` → Shows "processed"

---

12. **Customer creates validation** → `POST /validations/create` → API Gateway
13. **API Gateway calls Job Orchestrator** → `CreateValidationJob` (gRPC)
14. **Job Orchestrator queues job** → RabbitMQ message
15. **Worker picks up job** → Calls Validation Engine services

---

16. **Phase 1: Profiling** → Job Orchestrator → Validation Engine `ProfileDataset`
17. **Phase 2: Diversity** → Job Orchestrator → Validation Engine `AnalyzeDiversity`
18. **Phase 3: Pre-Screen** → Job Orchestrator → Validation Engine `PreScreenRisk`
    - Validation Engine → Collapse Engine `MatchSignatures` (gRPC)
    - Collapse Engine queries Signature Library Service
19. **Phase 4: Cascade** → Job Orchestrator → Validation Engine `TrainCascade` (streaming)
    - Validation Engine streams progress updates
    - Job Orchestrator → WebSocket broadcast to frontend
20. **Phase 5: Collapse Detection** → Validation Engine → Collapse Engine `DetectCollapse`

- If collapse detected → `LocalizeProblems` → `GenerateRecommendations`

21. **Phase 6: Report** → Job Orchestrator → Report Generator Service (gRPC)
22. **Job Orchestrator updates database** → Validation status = "completed"
23. **WebSocket pushes final results** → Frontend updates UI

6.2 Warranty Claim Flow

1. **Customer files claim** → `POST /warranties/{id}/claim` → API Gateway
2. **API Gateway calls Insurance Policy Service** → `CreateClaim` (gRPC)
3. **Insurance Policy Service validates claim** → Checks warranty terms, training logs
4. **Automated verification** → Technical review (Job Orchestrator → Validation Engine)
5. **Validation Engine analyzes training logs** → Compare actual vs predicted
6. **Collapse Engine determines root cause** → Data issue vs customer issue
7. **Insurance Policy Service calculates payout** → If approved
8. **Legal approval workflow** → External integration (email/task system)
9. **Payout processing** → Financial integration (Stripe/bank transfer)
10. **Frontend notification** → Email + in-app notification
11. **Update warranty status** → Database + frontend refresh

VII. ERROR HANDLING & EDGE CASES

7.1 API Error Responses

Standard Error Format:

```
json
```

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Validation job failed during cascade training",
    "details": {
      "validation_id": "val_ghi789",
      "stage": "cascade_training",
      "reason": "GPU memory exceeded during Tier 3 training",
      "retry_possible": true
    },
    "timestamp": "2025-10-23T12:00:00Z",
    "request_id": "req_abc123"
  }
}
```

HTTP Status Codes:

- `400 Bad Request` - Invalid input
- `401 Unauthorized` - Missing/invalid token
- `403 Forbidden` - Insufficient permissions
- `404 Not Found` - Resource doesn't exist
- `409 Conflict` - Resource already exists
- `422 Unprocessable Entity` - Valid format, invalid data
- `429 Too Many Requests` - Rate limit exceeded
- `500 Internal Server Error` - Server-side failure
- `503 Service Unavailable` - Service temporarily down

7.2 Retry Logic

Frontend Retry Strategy:

```javascript
```

```
async function apiCallWithRetry(url, options, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const response = await fetch(url, options);

      if (response.ok) {
        return await response.json();
      }

      // Retry on 5xx errors
      if (response.status >= 500 && i < maxRetries - 1) {
        await sleep(Math.pow(2, i) * 1000); // Exponential backoff
        continue;
      }

      // Don't retry on 4xx errors
      if (response.status >= 400 && response.status < 500) {
        throw new Error(`Client error: ${response.status}`);
      }

    } catch (error) {
      if (i === maxRetries - 1) throw error;
      await sleep(Math.pow(2, i) * 1000);
    }
  }
}
```

VIII. FRONTEND STATE MANAGEMENT

8.1 Context Providers

Auth Context:

```typescript
interface AuthContextType {
  user: User | null;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
  refreshToken: () => Promise<void>;
  isAuthenticated: boolean;
  isLoading: boolean;
}
```

Validation Context:

```typescript
interface ValidationContextType {
  validations: Validation[];
  activeValidation: Validation | null;
  fetchValidations: () => Promise<void>;
  createValidation: (request: CreateValidationRequest) => Promise<string>;
  subscribeToValidation: (validationId: string) => void;
  unsubscribeFromValidation: (validationId: string) => void;
}
```

8.2 Custom Hooks

useValidationProgress:

```typescript
function useValidationProgress(validationId: string) {
  const [progress, setProgress] = useState(0);
  const [currentStage, setCurrentStage] = useState('');
  const [status, setStatus] = useState<ValidationStatus>('queued');

  useEffect(() => {
    // WebSocket or polling logic
    const ws = connectToValidation(validationId);

    ws.onmessage = (event) => {
      const update = JSON.parse(event.data);
      setProgress(update.progress);
      setCurrentStage(update.current_stage);
      setStatus(update.status);
    };

    return () => ws.close();
  }, [validationId]);

  return { progress, currentStage, status };
}
```

IX. PERFORMANCE OPTIMIZATION

9.1 Frontend Optimizations

Lazy Loading:

- Route-based code splitting
- Component lazy loading for heavy charts
- Image lazy loading with placeholders

Caching:

- Cache dataset list (5 minutes)
- Cache completed validation results (indefinite)
- Invalidate cache on mutations

Pagination:

- Virtual scrolling for large lists
- Infinite scroll for validation history
- Server-side pagination (20 items/page)

Bundle Optimization:

- Tree shaking unused code
- Dynamic imports for chart libraries
- CDN for static assets

9.2 API Optimizations

Response Compression:

- gzip compression for all responses
- Brotli for static assets

Request Batching:

- Batch multiple dataset fetches
- Combine related API calls

Caching Headers:

- ETag for conditional requests
- Cache-Control for static resources

X. SECURITY CONSIDERATIONS

10.1 Authentication Flow

JWT Token Structure:

```json
{
  "sub": "usr_abc123",
  "email": "user@company.com",
  "company_id": "cmp_xyz789",
  "role": "admin",
  "tier": "professional",
  "iat": 1698067200,
  "exp": 1698070800
}
```

Token Refresh:

- Access token: 15 minutes expiry
- Refresh token: 30 days expiry
- Auto-refresh before expiry
- Secure HTTP-only cookies for refresh tokens

10.2 Data Security

Encryption:

- TLS 1.3 for all API calls
- AES-256 for data at rest
- End-to-end encryption for sensitive data

Access Control:

- Role-based permissions (user, admin, enterprise)
- Resource-level permissions (own datasets only)
- API key scoping (read-only, read-write)

Audit Logging:

- Log all data access
- Log all validation requests

- Log all warranty claims
- Retention: 7 years (compliance)

XI. DEPLOYMENT ARCHITECTURE

11.1 Frontend Deployment

Platform: Vercel or AWS CloudFront + S3 **CDN:** Global edge network **Build:** Next.js static generation + SSR **Environment:** Production, Staging, Development

11.2 Backend Deployment

Kubernetes Cluster:

- API Gateway: 3 replicas (autoscale to 10)
- Job Orchestrator: 2 replicas
- Data Service: 2 replicas
- Validation Engine: 5 replicas (GPU-bound)
- Collapse Engine: 3 replicas

Database:

- PostgreSQL (RDS): Multi-AZ, read replicas
- Redis (ElastiCache): Cluster mode

Storage:

- S3: Customer datasets (lifecycle policies)
- S3: Validation reports (archived after 1 year)

XII. MONITORING & OBSERVABILITY

12.1 Metrics to Track

Frontend:

- Page load time (<2s target)
- API call latency (<500ms target)
- Error rate (<0.1%)
- User engagement (pages/session)

Backend:

- Request throughput (requests/sec)
- API latency (p50, p95, p99)
- gRPC call duration
- Validation job completion time
- Error rates per service

12.2 Alerting

Critical Alerts:

- Service down (5+ minutes)
- Database connection failure
- GPU cluster unavailable
- Validation failure rate >5%

Warning Alerts:

- High API latency (>1s p95)
- Low disk space (<20%)
- High memory usage (>85%)
- Unusual error rate (>1%)

XIII. CONCLUSION

This complete API and frontend architecture provides:

✅ **Clear API contracts** - REST for customers, gRPC for internal services ✅ **Real-time updates** - WebSocket + polling fallback ✅ **Comprehensive frontend** - 11 core pages covering full workflow ✅ **Scalable design** - Microservices, async processing, caching ✅ **Security-first** - JWT auth, encryption, audit logs ✅ **Production-ready** - Error handling, monitoring, deployment

Next Steps:

1. Implement gRPC proto definitions
2. Build API Gateway with route handlers
3. Create frontend page components
4. Integrate WebSocket for live updates

5. Deploy to staging and test end-to-end

This architecture supports your billion-dollar validation platform. Now build it.